

$\text{SO}(3)$ Equivariant Neural Networks

A Five-Lecture Journal-Club Series

From physical symmetry to spherical tensor-product networks



Author

Phong Dang

Prepared with AI assistance (ChatGPT and Claude)

September 2026

Abstract

Physical predictions do not respond arbitrarily when a system is rotated. Energies remain invariant; forces, dipoles, and torques rotate as vectors; quadrupoles and stresses follow higher-rank transformation laws. Equivariant neural networks encode these facts in their architecture rather than asking a generic model to rediscover them from augmented data.

This five-lecture series develops an $SO(3)$ -equivariant neural network from first principles for an audience new to group theory, Clebsch–Gordan coefficients, and equivariant learning. The route is deliberately physical: begin with transformation laws, interpret irreducible representations as geometric feature types, understand Clebsch–Gordan coefficients as a fixed change of basis for combining types, and then assemble channel mixing, tensor products, and scalar gates into a trainable network. Each lecture contains parallel mathematics, physics, and AI perspectives, a participant notebook, discussion pauses, and a separate solution note.

The series culminates in a small spherical-basis network that learns a vector-valued response from two input vectors and generalizes across unseen orientations by construction. The final lecture connects this example to equivariant particle models and neural quantum states, where symmetry can constrain a variational wavefunction to a desired angular-momentum sector.

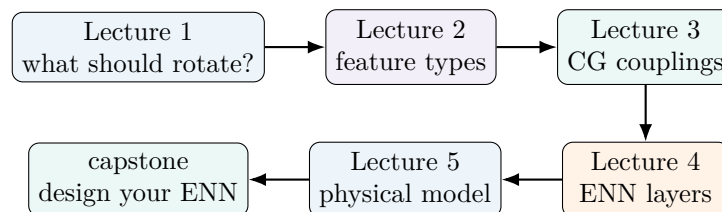
Key idea

The series goal is practical conceptual fluency: participants should finish able to sketch an $SO(3)$ ENN for a physical system, explain why it is equivariant, and design numerical tests that could falsify that claim.

1 The big picture

The conceptual arc is

physical law $\longrightarrow$ transformation type $\longrightarrow$ allowed couplings
 $\longrightarrow$ equivariant architecture $\longrightarrow$ physical prediction.



The course uses one recurring target,

$$y(u, v) = (u \cdot v)(u \times v), \quad (1)$$

because it combines an invariant scalar with an equivariant vector. The simplicity keeps attention on architecture rather than domain-specific preprocessing.

2 Guide to the five lectures

Lecture 1: What Should Rotate?

Central question: How should a physical prediction change when the complete experiment is rotated?

The lecture introduces $\text{SO}(3)$, invariance, equivariance, and the commuting-diagram test. Dot and cross products provide concrete examples. The notebook generates rotations and numerically distinguishes an equivariant function from an axis-dependent one.

Takeaway: symmetry specifies an output transformation law, not necessarily an unchanged output.

Lecture 2: Features Have Geometric Types

Central question: How does the same physical rotation act on scalars, vectors, and higher angular features?

The lecture introduces representations, irreps $\ell = 0, 1, 2$, spherical components $m = -\ell, \dots, \ell$, and Wigner D matrices. The notebook performs Cartesian/spherical round trips and verifies the representation law.

Takeaway: ℓ is a geometric type, m labels components inside the type, and channels count learned copies of that type.

Lecture 3: How Geometric Types Combine

Central question: How can two rotating quantities be combined into a new quantity with a definite transformation type?

The lecture develops tensor products, the triangle rule, and the decomposition $1 \otimes 1 = 0 \oplus 1 \oplus 2$. CG coefficients are presented as a change-of-basis table and tested as intertwiners. Participants never need to derive the coefficients by hand.

Takeaway: fixed CG tensors sort products into valid output irreps.

Lecture 4: From Couplings to a Neural Network

Central question: Which learned operations preserve the representation structure?

The lecture assembles same- ℓ channel mixing, fixed CG tensor products, and invariant scalar gates. The notebook constructs an untrained network that is already equivariant, then breaks each of the four architectural rules in turn and reports all of them as a single table.

Takeaway: learned weights act on channels; symmetry-fixed operators act on magnetic indices.

Lecture 5: Build a Physical $\text{SO}(3)$ ENN

Central question: How do we train, validate, and extend the architecture to a physical system?

The lecture trains the two-vector network on restricted orientations, compares it with a Cartesian MLP, and separates prediction error from equivariance error. It then asks whether data augmentation is an adequate substitute, ablates the learned coupling paths to see which the trained model actually relies on, and sketches a many-particle message-passing model and an NQS extension.

Takeaway: equivariance improves orientation generalization but remains only one component of a physically sound model.

3 How to run each 90-minute session

Every lecture follows a stable rhythm:

Minutes	Mode	Purpose
0–10	physical hook	surface intuition and misconceptions
10–30	theory	introduce one mathematical structure
30–45	physics example	interpret the structure in a familiar system
45–50	break	allow consolidation
50–70	AI translation	turn the structure into architecture
70–83	notebook or group task	test the claims computationally
83–90	discussion and exit ticket	synthesize and diagnose understanding

The notes are written to support live teaching, not silent reading alone. The colored lenses can be assigned to different discussants:

- **Math lens:** What identity or structure guarantees the statement?
- **Physics lens:** What quantity or conservation principle does it describe?
- **AI lens:** Which architectural choice follows from it?

4 Package contents and recommended use

For each lecture, the package provides:

- a self-contained illustrated lecture note with no table of contents;
- a solution PDF answering all numbered questions and notebook reference checks;
- a runnable participant notebook with demonstrations and optional exercises.

Recommended participant preparation is light: review the previous lecture’s checkpoint and spend no more than 20–30 minutes on the optional notebook exercises. No prior group theory is assumed. Basic linear algebra, vectors, and familiarity with Python/NumPy are helpful.

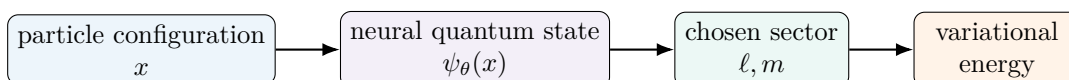
The original `ENN_S03_intro.ipynb` remains the technical anchor. The lecture notebooks isolate one conceptual layer at a time, while the original notebook shows the complete progression in one place.

5 A thread toward neural quantum states

A neural quantum state replaces an explicit many-body wavefunction table with a parameterized model $\psi_\theta(x)$ and optimizes

$$E_\theta = \frac{\langle \psi_\theta | H | \psi_\theta \rangle}{\langle \psi_\theta | \psi_\theta \rangle}. \quad (2)$$

The symmetry question is then: in which representation should the trial state transform?



A scalar rotational ground state may be modeled in an $\ell = 0$ sector. Excited states can form $\ell > 0$ multiplets with components m . Symmetry may be encouraged softly through penalties involving the angular-momentum Casimir, or enforced structurally by building the representation type into the network. The latter mirrors the central philosophy of this series.

This connection should remain a motivating horizon rather than a detour. Fermionic antisymmetry, Monte Carlo sampling, nodal structure, and variational optimization introduce additional challenges that lie beyond these five lectures.

6 What success looks like

At the end, a participant should be able to look at a proposed physical learning problem and answer:

1. What transformations define the symmetry?
2. How does each input and output transform?
3. Which irrep types should hidden features carry?
4. Which tensor-product paths are permitted?
5. Where do fixed CG tensors and learned weights act?
6. Which nonlinearities preserve equivariance?
7. What numerical experiment could expose a symmetry bug?

Key idea

The intended endpoint is not mastery of abstract representation theory. It is the ability to use a small amount of representation theory as a reliable engineering language for physical neural networks.

Credits and acknowledgements

Author: Phong Dang.

The series was developed as a teaching companion to the SO(3) equivariant neural-network notebook. Phong Dang led the pedagogical direction and journal-club design and is responsible for the content.

The material was prepared with substantial assistance from two AI systems. ChatGPT contributed to the curriculum structure, technical exposition, code, and document production; Claude contributed technical review, corrections, and the additional numerical experiments in Lectures 4 and 5. Both are acknowledged here rather than listed as authors, because authorship carries accountability that only a person can hold.